



HARAMAYA UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING

Software Design Document (SDD) for Cafeteria Management System (HUcMS)

Prepared by:

NAME: IDNo:

Amenti Liben(0686/16)

Boka Jireгна(0931/16)

Chala Gobana..... (0964/16)

Damoze Motuma(0995/16)

Dawit Marsha..... (1026/16)

Jalane Feyisa..... (1663/16)

Submitted to: Mr.Shambel Adugna
Date of Submission: April 22, 2026

Version: 1.0

TABLE OF CONTENTS

1. Introduction
 - .1 Purpose
 - 1.2 System Overview
 - 1.3 Design Objectives
 - 1.4 References
 - 1.5 Definitions and Acronyms
2. Design Overview
 - 2.1 Design Approach
 - 2.2 Environment Overview
 - 2.3 System Architecture
 - 2.4 Constraints and Assumptions
3. Interfaces and Data Stores
 - 3.1 System Interfaces
 - 3.2 External System Interfaces
 - 3.3 Data Stores
4. Structural Design
 - 4.1 Design Patterns Used
 - 4.2 Class Diagram Notation
 - 4.3 Class Descriptions
5. Behavioral Models
 - 5.1 Activity Diagram Notation
 - 5.2 State Machine Diagram Notation
 - 5.3 Sequence Diagram Notation
6. Implementation-Level Models
 - 6.1 Package Diagram Notation
 - 6.2 Component Diagram Notation
 - 6.3 Deployment Diagram Notation
7. User Interface Design
8. Database Design
9. Error Handling
10. Security Design
11. Performance Design
12. Conclusion

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) provides a detailed architectural, structural, and behavioral design of the **Haramaya University Cafeteria Management System (HUcMS)**. It translates the **SRS requirements** into a practical, implementable design using **UML diagrams** including class, sequence, activity, state machine, package, component, and deployment diagrams.

1.2 System Overview

The **HUcMS** automates student meal verification, eligibility checking, meal logging, and reporting. Students use **facial recognition** or **ID card** at cafeteria gates. The system checks the meal period (breakfast, lunch, or dinner), daily quota, and prevents duplicate entries. Staff and administrators manage configurations and view reports.

The system operates as follows:

- **Students** approach the cafeteria gate and authenticate using facial recognition or ID card.
- The system verifies the student's eligibility for the meal period. □ It logs the meal and checks the daily quota.
- **Staff/admins** manage configurations, generate reports, and monitor system performance.

1.3 Design Objectives

- Automate meal verification and logging.
- Ensure accurate tracking of meal quotas.
- Prevent duplicate meal entries.
- Provide a secure and efficient system for students and staff. □ Support scalability and future enhancements.

1.4 References

- SRS Document (Cafeteria Management System)
- UML 2.5 Specification
- Java Programming Language (Oracle)
- System Architecture Design Guidelines

1.5 Definitions and Acronyms

- **HUcMS**: Haramaya University Cafeteria Management System
- **SRS**: Software Requirements-ML **: Unified Modeling Language SD Software Design Document

2 Design Overview

2.1 Design Approach

The design follows an **Object-Oriented Design (OOD)** methodology combined with a **ThreeTier Architecture** pattern.

Object-Oriented Design was chosen because the problem domain naturally maps to objects such as **Student**, **MealRecord**, and **MealPeriod**. It promotes code reuse through inheritance and polymorphism. Encapsulation enhances security by hiding implementation details. The system requires complex relationships that are easily modeled with OOD.

The **Three-Tier Architecture** separates the system into three distinct tiers:

- **Presentation Tier:** This tier handles user interaction and displays information. It includes the **Student Portal**, **Staff Dashboard**, **Admin Panel**, and **Gate Interface**. These components provide the user-facing interfaces for students, staff, and administrators to interact with the system.
- **Business Logic Tier:** This tier implements core business rules and processing. It contains services such as the **Authentication Controller**, **Meal Session Manager**, **Report Generator**, **Audit Service**, **Student Service**, and **Configuration Service**. These components logic of the system, ensuring that business and data is processed efficiently.
- **Data Tier:** This tier manages persistent data storage and retrieval. It includes the **MySQL Database** for storing all system data, **Redis** for caching frequently accessed data, and **Backup Storage** for disaster recovery. This ensures data integrity, performance, and reliability.

This separation provides **scalability**, **maintainability**, **security**, and **reusability**. Each tier can be scaled independently, changes to one tier do not affect others, business logic and data are hidden from end users, and business logic can be reused by different presentation interfaces.

This design approach ensures that the **Haramaya University Cafeteria Management System (HUcMS)** is **modular**, **extensible**, and **maintainable**, with a strong foundation for future enhancements and system expansion.

2.2 Environment Overview

The **Heteria Management System (HUcMS)** is designed to operate in a **multi-tiered, distributed environment** that supports both **physical and software components**. The system is built to be **scalable**, **secure**, and **efficient**, with a focus on **real-time data processing** and **user-friendly access**.

Physical Environment

- **Central Server:** The system is hosted on a **Ubuntu 22.04 LTS server** located in the **university data center**. This server acts as the **central hub** for all system operations, including data processing, user authentication, and system management.

- **Cafeteria Gate Workstations:** At each cafeteria entrance, there are **workstations** equipped with **network cameras and ID card readers**. These devices are connected via **LAN** to the central server, enabling real-time data capture and processing of student meal access.
- **Student Access Points:** Students can access the system through the **HU E-learning portal** via **WiFi**. The portal the central server through an **API**, allowing seamless integration with the cafeteria management system.
- **Network Infrastructure:** The system relies on a **secure and reliable LAN network** to connect all components. This ensures low-latency communication between the **gate workstations, central server, and student devices**.
- **Student Devices:** Students can access the system through **university computer labs** or **personal devices** (such as laptops, tablets, or smartphones) that are connected to the university's WiFi network.

Software Environment

- **Operating Systems:**
- **Server: Ubuntu 22.04 LTS** (long-term support, stable, and secure).
- **Workstations: Windows 10** (for user interfaces and local processing).
- **Web Server:**
- **Python: Nginx with Gunicorn** (for deploying Django-based applications). □ **PHP: Apache** (for deploying Laravel-based applications).
- **Database:**
- ****MySQL** for persistent storage of system data, including student records, meal sessions, and audit logs).
- **Application Framework:**
- **Python: Django** (a high-level, full-featured web framework).
- **PHP: Laravel** (a modern, expressive PHP framework).
- **Client Browsers:**
- **Chrome, Firefox, and Microsoft Edge** (for accessing the student portal, staff dashboard, and admin panel).

Network and Communication

- The system uses **API-based communication** between the **student portal** and the **central server**, ensuring secure and efficient data exchange.
- **Docker containers** are used to deploy and manage the system components, ensuring consistency across development, testing, and production environments.
- **Redis** is used for **caching** frequently accessed data, improving system performance and reducing database load.
- **Backup Storage** is implemented to ensure data **integrity and disaster recovery**.

This environment design ensures that the **** is **robust, scalable, and adaptable** to future requirements. It supports **real-time processing, secure access, and efficient data**

management, making it an ideal solution for the **Haramaya University Cafeteria Management System**.

2.3 System Architecture

The **Haramaya University Cafeteria Management System (HUcMS)** is designed using a **Three-Tier Client-Server Architecture**, which ensures **modularity, scalability, and maintainability**. This architecture separates the system into three distinct layers: **Presentation Tier**, **Business Logic Tier**, and **Data Tier**, each responsible for specific functions to ensure efficient and secure operation.

1. Three-Tier Architecture Overview

Presentation Tier

The **Presentation Tier** is the **user-facing layer** of the system, responsible for **user interaction** and **data display**. It includes the following components:

- **Student Portal**: A web-based interface for students to view their **meal history, profile information, and meal eligibility**.
- **Staff Dashboard**: A web-based interface for staff members to **monitor daily operations, view real-time data, and manage meal sessions**.
- **Admin Panel**: A web-based interface for administrators to **perform full system administration**, including **user management, system configuration, and audit logging**.
- **Gate Interface**: A **real-time interface** at cafeteria gates that displays **scan results, meal eligibility, and transaction status** for students.

This tier ensures **user-friendly access** and **real-time interaction** with the system.

Business Logic Tier

The **Business Logic Tier** is the **core of the system**, responsible for **processing business rules, validating data, and managing system operations**. It includes the following components:

- **Authentication Controller**: Handles **user authentication** and **session management** for students, staff, and administrators.
- **Meal Session Manager**: Manages **meal periods, meal quotas, and eligibility rules** for students.
- **Report Generator**: Generates **analytical reports** from transaction data, including **meal usage statistics, student behavior analysis, and system performance metrics**.
- **Audit Service**: Logs all **system events** for **security, debugging, and compliance** purposes.
- **Student Service**: Manages **student data retrieval, updates, and profile management**.
- **Configuration Service**: Manages **system settings, meal periods, and user permissions**.

This tier ensures **secure, efficient, and accurate processing** of all system operations.

Data Tier

The **Data Tier** is responsible for **persistent data storage**, **data retrieval**, and **data management**. It includes the following components:

- **MySQL Database:** A **relational database** used for **persistent storage** of all system data, including **student records**, **meal transactions**, **audit logs**, and **system configurations**.
- **Backup Storage:** An **automated backup system** that ensures **data integrity** and **disaster recovery**.
- **Cache Layer (Redis):** A **distributed in-memory cache** used to **improve performance** by caching frequently accessed data, such as **student profiles** and **meal session details**. This tier ensures **data consistency**, **security**, and **high availability**.

2. Major Subsystems

is composed of four, each responsible for a specific function:

1. Student Verification Subsystem

- **Function:** Captures and validates **student identification** (e.g., ID cards, biometric data, or student numbers).
- **Purpose:** Ensures that only **authorized students** can access meal services.

2. Meal Management Subsystem

- **Function:** Enforces **meal periods**, **meal quotas**, and **eligibility rules**.
- **Purpose:** Ensures that students only access meals during **allowed times** and **within their allocated quotas**.

3. Transaction Logging Subsystem

- **Function:** Records all **meal transactions**, including **scan results**, **meal types**, and **transaction timestamps**.
- **Purpose:** Provides a **complete audit trail** for **system accountability** and **data analysis**.

4. Reporting Subsystem

- **Function:** Generates **analytical reports** based on **transaction data**.
- **Purpose:** Supports **decision-making**, **resource planning**, and **system optimization**

5. User Management Subsystem

- **Function:** Manages **staff and admin accounts**, including **user roles**, **permissions**, and **access control**.
- **Purpose:** Ensures **secure access** and **proper system administration**.

2.4 Constraints and Assumptions

The **Haramaya University Cafeteria Management System (HUcMS)** is designed with a set of **constraints and assumptions** to ensure **system reliability, security, and compliance** with university standards and operational requirements.

3. Interfaces and Data Stores

3.1 System Interfaces

The **Haramaya University Cafeteria Management System (HUcMS)** interacts with various **hardware and software interfaces** to ensure **accurate student identification, secure data exchange, and seamless integration with university systems**. Below are the key interfaces and their specifications.

3.1.1 Biometric Camera Interface

Function

This interface is responsible for **capturing live facial images** of students for **identity verification**. It serves as a **hardware-software interface** between the **camera device** and the **system application**.

Technical Details

- **Connection Type:** USB or IP camera.
 - **Protocol:** Uses a **camera SDK** (e.g., OpenCV or similar) to provide **frames** for processing.
 - **Data Format:**
 - **Raw Image:** JPEG or PNG format.
 - **Processed Data:** **Feature vector of 128 bytes** (face embedding).
 - **Operations:**
 - `captureFrame()`: Captures a single frame from the camera.
 - `detectFace()`: Detects a face in the image.
 - `extractFeatures()`: Extracts the face embedding vector.
 - `matchTemplate()`: Compares the feature vector with stored templates.
 - **Error Handling:**
 - Retry connection if the camera is not detected.
 - Log an error and display a message if no face is detected. □ Display a warning if lighting conditions are poor.
-

3.1.2 ID Card Reader Interface

Function

This interface is used to **read student ID cards** via **NFC, RFID, or barcode** for **student identification**.

Technical Details

- **Connection Type:** USB HID keyboard wedge or serial device.

-: **Reads the.**

Data Format**: **10-digit student ID string.**

- **Operations:**
- `initializeReader()`: Initializes and connects to the reader.
- `readCard()`: Waits for a card and reads the ID.
- `cancelRead()`: Cancels the waiting operation.
- **Error Handling:**

- Display an error and attempt reconnection if the reader is not connected.
- Return to idle state if the card read times out.
- Display an invalid card message if the card format is invalid.

3.1.3 QR Code Scanner Interface

Function

This interface is used to **scan QR codes** displayed on **student phones or ID cards** for **student identification and transaction verification**.

Technical Details

- **Connection Type:** USB camera with QR decoding library (e.g., ZBar or similar).
- **Protocol:** Uses a **QR decoding library** to capture and decode the QR code.
- **Data Format:** **JSON string** containing:
- Student ID □ Timestamp □ **Operations:**
- **startScanner():** Initializes the scanner.
- **scanQR():** Captures and decodes the QR code.
- **stopScanner():** Releases camera resources.
- **Error Handling:**
- Retry and display guidance if the QR code is not detected.
- Display an expired or invalid QR message if the code is not valid.

3.2 External System Interfaces

Function

The system also interacts with **external systems** such as the **SIMRS database**, **E-learning portal**, and **university ID systems** for **data exchange and integration**.

Technical Details

- **SIMRS Database:**
- **Access Type:** Read-only.
- **Purpose:** Retrieve student information (name, ID, enrollment status).
- **E-learning Portal:**
- **Access Type:** API-based.
- **Purpose:** Retrieve student meal eligibility and transaction history.
- **University ID Systems:**
- **Access Type:** API-based.
- **Purpose:** Validate student ID cards and ensure data consistency.

3.3 Data Stores

The **HUcMS system** uses a **centralized data store** to manage **student data, transaction logs, and system configurations**. The data store is designed to ensure **data integrity, security, and scalability**.

Data Store Details

- **Database: MySQL** (as per university standard).
- **Tables:**
 - students: Stores student information (ID, name, enrollment status, etc.).
 - transactions: Logs cafeteria transactions (, meal type, etc.).
 - configurations: Stores system

1. Hardware Constraints

- **Verification Devices Compatibility:** All verification devices (e.g., biometric scanners, RFID readers) must be **compatible with the workstation operating system** (e.g., Windows, Linux).
- **USB Port Availability:** Workstations must have **USB ports** to connect verification devices for data input.
- **Network Latency:** The **network latency** between gate workstations and the central server must be **less than 50 milliseconds** to ensure **real-time transaction processing** and **accurate scan results**.

2. Software Constraints

- **Open-Source Technologies:** The system must be built using **open-source technologies** to avoid **proprietary software licensing costs** and ensure **flexibility and maintainability**.
- **Database Requirement: MySQL** must be used as the **database system**, as it is the **university's standard database** and is well-suited for the system's **transactional and analytical needs**.
- **Containerization:** The system must be **deployable using Docker containers** to consistent environment setup**, **easy scaling**, and **portability across different deployment environments**.

3. Integration Constraints

- **SIMRS Database Read-Only:** The **Student Information Management and Registration System (SIMRS)** database is **read-only**, so the **HUcMS system cannot modify student enrollment data**. All student data must be **read-only and derived from SIMRS**.

- **E-Learning Portal Integration:** The E-learning portal integration must use **existing APIs** without modification to ensure **compatibility and minimal development effort**.

4. Security Constraints

- **HTTPS with TLS 1.3:** All communication between the system and external services must use **HTTPS with TLS 1.3** to ensure **secure data transmission** and **** of eavesdropping****.
- **Biometric Data Storage:** **Biometric data** (e.g., facial recognition, fingerprint) must be **stored as encrypted feature vectors**, not as ********, to ensure **privacy and data security**.
- **Password Hashing:** Passwords hashed using **bcrypt or Argon2** to ****** and **resistance to brute-force attacks**.

5. Assumptions

- **SIMRS Database Accuracy:** The **SIMRS database** is populated with **accurate, up-to-date student information**, ensuring that **student data is reliable** for the HUcMS system.
- **Network Infrastructure:** The **university's network infrastructure is reliable and maintained by the university IT department**, ensuring **consistent and secure connectivity**.
- **Valid University ID Cards:** Students have **valid university ID cards with readable barcodes or RFID chips**, ensuring **accurate student identification** at the cafeteria gates.
- **Stable Meal Period Definitions:** **Meal period definitions** (e.g., lunch, dinner, snack times) are **stable and consistent**, ensuring **accurate meal eligibility checks**.
- **User Literacy:** Users (students, staff, and administrators) have **basic computer literacy**, ensuring **effective use of the system** and **minimal training requirements**.

This section ensures that the **HUcMS system is built within the technical, operational, and security boundaries** of the university environment, while maintaining **scalability, security, and compliance** with institutional standards.

4. Structural Design

The **Haramaya University Cafeteria Management System (HUcMS)** is built using a **modular, scalable, and maintainable architecture** that leverages **design patterns** to ensure **separation of concerns, loose coupling, and reusability**. Below is a detailed breakdown of the **design patterns** used in the system.

4.1 Design Patterns Used

4.1.1 Singleton Pattern

Used in: MealSessionManager

Purpose: Ensures that only **one meal session** is active at any time, preventing concurrency issues and ensuring system consistency.

Implementation Details:

- A **private static instance** of the class is maintained.
- A **private constructor** prevents external instantiation.
- A **public static getInstance()** method provides access to the single instance. **Benefits:**
- Ensures **single instance control** for meal session management. □ Prevents **multiple instances** from interfering with each other.

4.1.2 Factory Pattern

Used in: VerificationHandlerFactory

Purpose: Provides a **common interface** for creating different verification handlers (Face, Card, QR) without exposing the **implementation details**.

Implementation Details:

- A **createHandler()** method returns an instance of FaceHandler, CardHandler, or QRHandler based on the input type.
- The factory abstracts the **creation logic** and allows **easy extension** of new verification methods. **Benefits:**
- **Decouples** the client code from the **specific verification implementations**. □ Simplifies **code maintenance** and **extension** of new verification types.

4.1.3 Strategy Pattern

Used in: EligibilityChecker

Purpose: Allows **eligibility rules** to be **changed dynamically** based on **meal period** or **student type**.

Implementation Details:

- An **EligibilityStrategy** interface defines the **common method** for checking eligibility.
- Concrete strategies (BreakfastStrategy, LunchStrategy, DinnerStrategy) implement the logic for each meal period.
- A **context class** (e.g., ElChecker) selects the appropriate strategy at runtime. **Benefits:**
- **Flexible** and **configurable** eligibility rules.
- **Decouples** the logic from the **client code** that uses it.

4.1.4 Observer Pattern

Used in: AuditService

Purpose: Enables **event logging** without **tight coupling** between the **event source** and **loggers**.

Implementation Details:

- An **Observable** interface defines methods for **attaching/detaching observers** and **notifying them**.
- AuditService implements the **Observer** interface to log events.
- System components (e.g., MealSessionManager, VerificationHandler) **notify observers** when events occur. **Benefits:**
- **Decouples** event sources from loggers.
- Allows **dynamic addition/removal** of observers.
- Supports **event-driven architecture** for logging and monitoring.

4.1.5 Repository Pattern

Used in: Database Access

Purpose: Abstracts **database access** to **reduce coupling** between the **business logic** and **data access**.

Implementation Details:

- StudentRepository, MealRecordRepository, and StaffRepository classes handle **CRUD operations** for their respective entities.
- Business logic uses **repositories** instead of direct SQL queries.
- Repositories encapsulate **data access logic**, **query building**, and **transaction management**.

Benefits:

- **Decouples** the **business logic** from the **database**.
- Simplifies **testing** and **maintenance** of data access code.

- Supports **easy switching** of data sources (e.g., from MySQL to PostgreSQL).

4.1.6 Dependency Injection Pattern

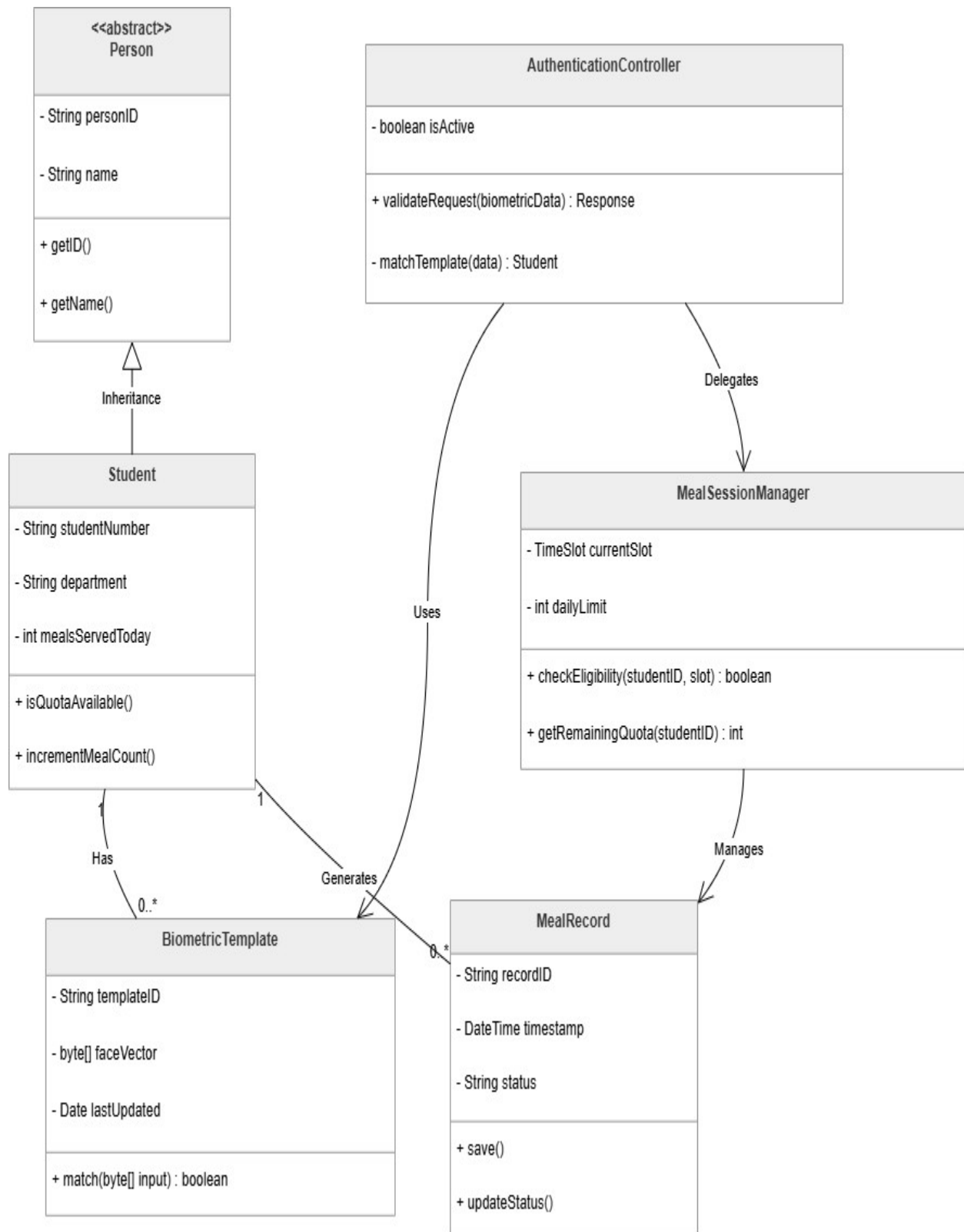
Used in: All Services

Purpose: Promotes **testability**, **modularity**, and **loose coupling** by **injecting dependencies** rather than hard-coding them.

Implementation Details:

- Dependencies are injected through **constructors** or **setter methods**.
- A **DI container** (e.g., Spring, Dagger, or a custom container) manages object creation and dependency resolution.
- **Mock objects** can be injected for **unit testing**. **Benefits:**
 - **Easier testing** with **mock dependencies**.
 - **Flexible** and **maintainable** code structure.
 - **Supports extensibility** and **modular design**.
 -

4.2 Class Diagram Notation



UML Elements Explained

1. Classes

Class Name	Description
Person	Abstract base class for all users (students, staff).
Student	Inheriting from Person, represents a student with biometric data.
StaffAccount	Inheriting from Person, represents a staff member with account status.
AuthenticationController	Manages authentication and verification processes.
MealSessionManager	Singleton class managing meal sessions and associated data.
MealPeriod	Represents a specific meal period (e.g., breakfast, lunch).
MealRecord	Stores records of student meals, including status and verification result.
BiometricTemplate	Stores biometric data for a student.
ReportGenerator	Generates reports in different formats (PDF, Excel, CSV, JSON).
AuditService	Logs events and maintains audit trails.
VerificationResult	Value object representing the result of a verification process.

4.3 Class Descriptions Person

Class (Abstract)

- **Description:** The Person class is an abstract class that serves as the base class for all persons in the system, including students and staff. It encapsulates common attributes and behaviors that are shared among all users. Since it is abstract, it cannot be instantiated directly and must be subclassed.
- **Attributes:**
 - **personID:** A unique identifier for the person.
 - **name:** The full name of the person.
 - **email:** The email the person.
 - **phone:** The phone number of the person.
- **Methods:**
 - **getPersonID():** Returns the person ID.
 - **getName():** Returns the person's name.
 - **getEmail():** Returns the person's email.
 - **getPhone():** Returns the person's phone number.
 - **setName():** Sets the person's name.

- setEmail(): Sets the person's email. □ setPhone(): Sets the person's phone number.

Student Class

- **Description:** The Student class represents a student who uses the cafeteria system. It extends the Person class and includes attributes specific to students, such as meal quota and enrollment information.
- **Attributes:**
 - studentNumber: The student number, which must follow the university format.
 - department: The department the student belongs to.
 - enrollmentStatus: The current enrollment status of the student.
 - mealPlanType: The type of meal plan the student is enrolled in.
 - dailyQuota: The maximum number of meals the student can take per day (cannot exceed 5).
 - mealsServedToday: The number of meals the student has already taken today.
- **Methods:**
 - getStudentNumber(): Returns the student number.
 - getDepartment(): Returns the department. □ getEnrollmentStatus(): Returns the student's enrollment status.
 - isQuotaAvailable(): Checks if the student has remaining meal quota.
 - incrementMealCount(): Increments the number of meals served today.
 - resetDailyQuota(): Resets the daily meal count to zero.
 - getRemainingMeals(): Returns the number of remaining meals for the day.
 - isValidEnrollment(): Validates whether the student is enrolled in the system.

AuthenticationController Class

- **Description:** The AuthenticationController class is the central controller for all verification attempts in the system. It implements both the **Factory** and **Singleton** patterns, ensuring that only one instance of the class exists at any time.
- **Attributes:**
 - faceMatcher: Handles facial recognition verification.
 - cardReader: Handles card-based verification.
 - qrScanner: Handles QR code-based verification.
 - static instance: The singleton instance of the class.
- **Methods:**
 - getInstance(): Returns the singleton instance of the class.
 - verifyFace(): Verifies a student's identity using facial recognition. □ verifyCard(): Verifies a student's identity using a card. □ verifyQR(): Verifies a student's identity using a QR code.
 - matchBiometric(): Matches biometric data for verification. □ validateStudentID(): Validates a student's ID.
 - getStudentFromID(): Retrieves a student based on their ID.

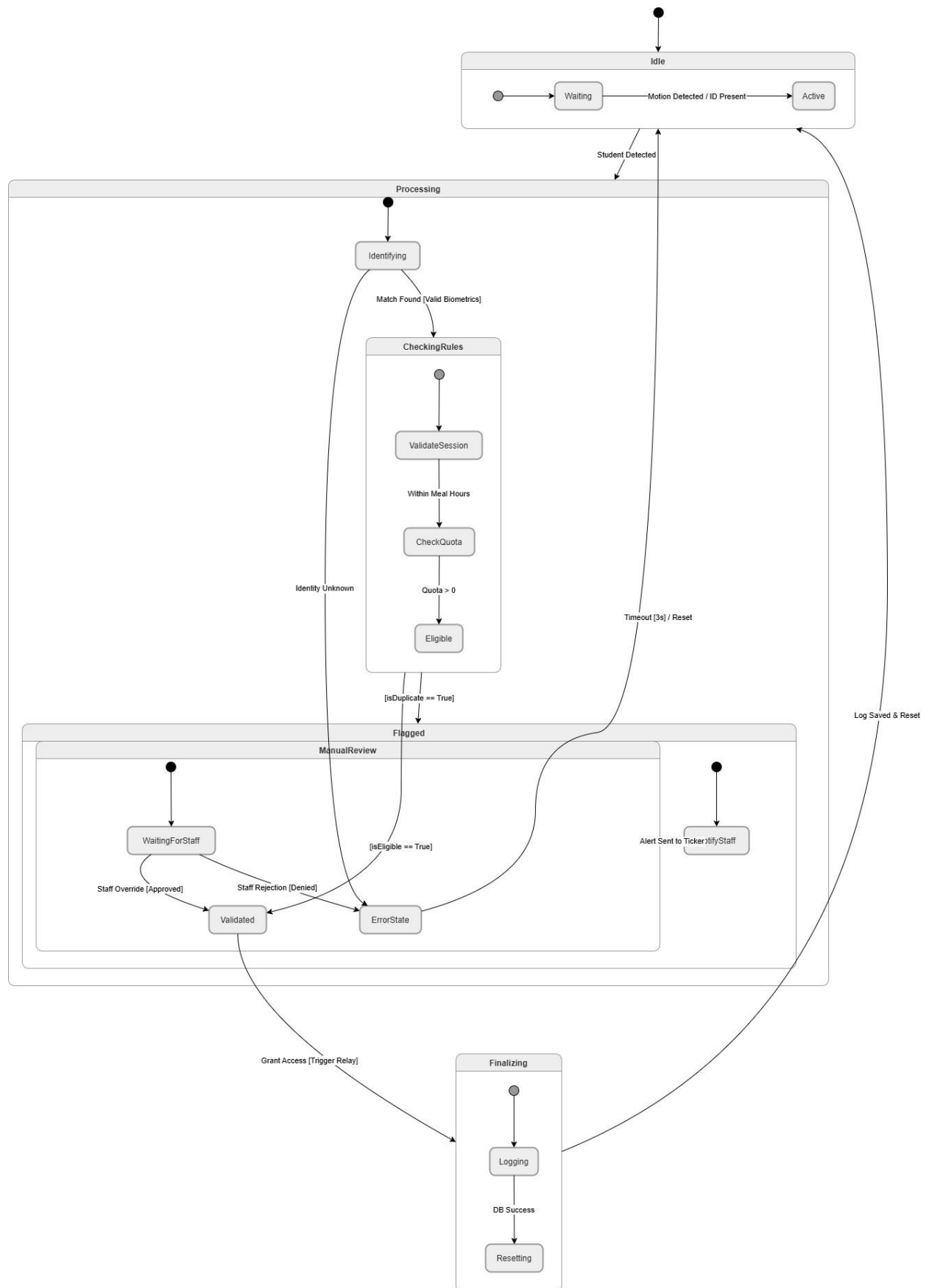
MealSessionManager Class

- **Description:** The MealSessionManager class manages meal periods, eligibility rules, and quota checking. It implements the **Singleton** pattern, ensuring that only one instance of the class exists.
- **Attributes:**

- `currentPeriod`: The current meal period.
- `periods`: A list of all meal periods.
- `dailyLimit`: The daily meal limit for students.
- `sessionActive`: A flag indicating whether the meal session is active. □ static
- `instance`: instance of the class.
- **Methods:**
- `getInstance()`: Returns the singleton instance of the class.
- `checkEligibility()`: Checks if a student is eligible to eat during the current period.
- `getRemainingQuota()`: Returns the remaining meal quota for the student.
- `getCurrentPeriod()`: Returns the current meal period.
- `isWithinMealHours()`: Checks if the current time falls within the meal period.
- `hasDuplicateEntry()`: Checks if a duplicate meal entry exists.
- `loadMealPeriods()`

5.2 Behavioral Models

5.2.1 Behavioral Diagrams



This diagram shows the **state transitions** a student goes through when attempting to record a meal. It includes:

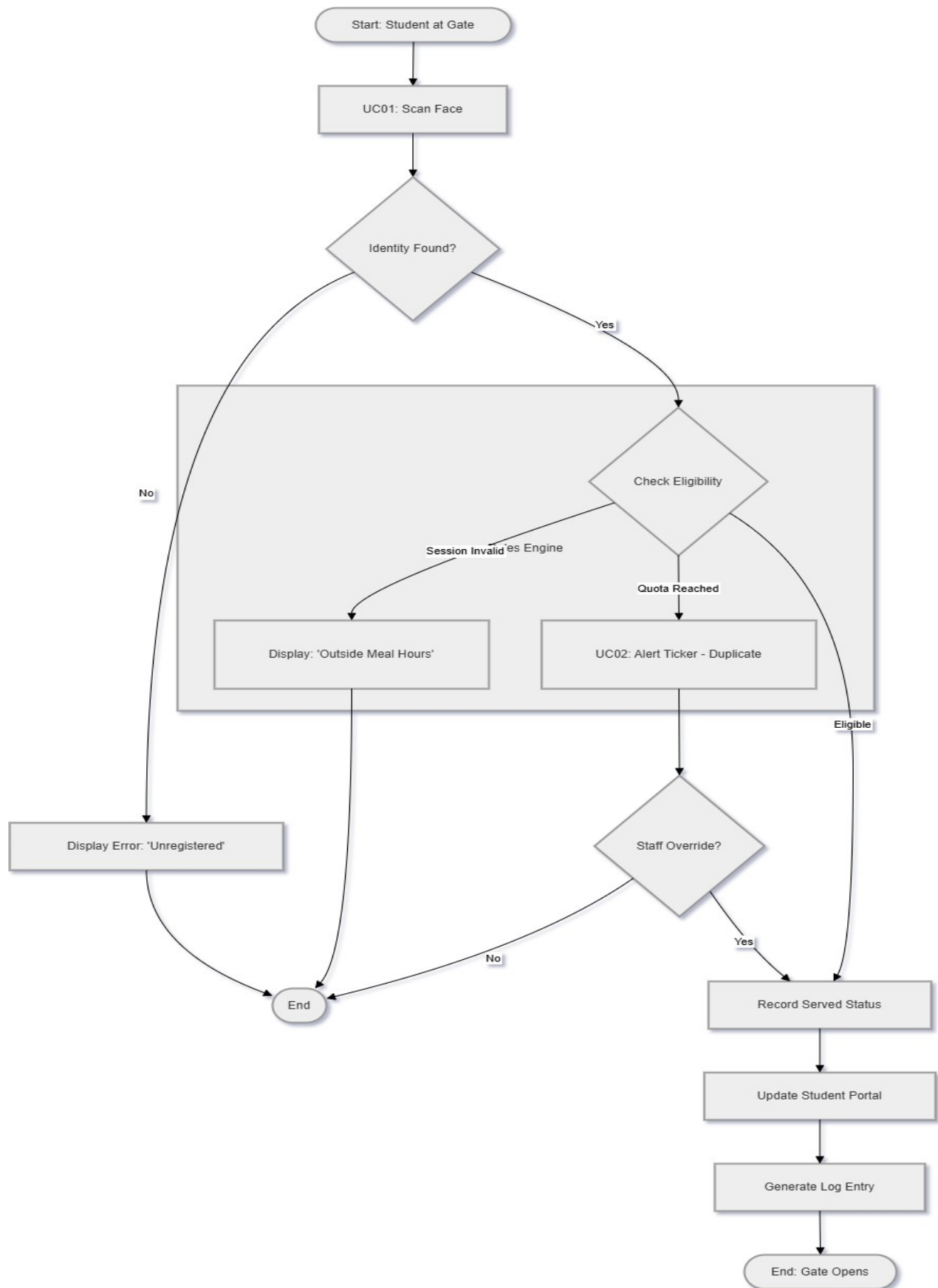
- **Student detection**
- **Biometric verification**
- **Meal period validation**
- **Quota check**
- **Duplicate check**
- **Staff override**
- **Final validation and logging**

Explanation of Key States and Transitions

State	Description	Transition
Idle	The system is waiting for a student to enter the cafeteria.	→ Waiting for Motion / ID Present
Waiting for Motion / ID Present	The system is waiting for a student to move or present an ID.	→ Student Detected
Student Detected	The system detects a student entering the area.	→ Processing
Processing	The system begins processing the student's entry.	→ Identifying
Identifying	The system identifies the student using biometric data.	→ Match Found / Identity Unknown
Match Found [Valid Biometrics]	The system matches the student's biometric data.	→ Checking Rules
Checking Rules	The system checks if the student is eligible for a meal.	→ Validate Session
Validate Session	The system checks if the current time is within a valid meal period.	→ Within Meal Hours
Within Meal Hours	The student is within the valid meal time window.	→ Check Quota
Check Quota	The system checks if the student has exceeded their meal quota.	→ Quota > 0 / Quota == 0
Quota > 0	The student has not exceeded their meal quota.	→ Eligible
Eligible	The student is eligible to record a meal.	→ Is Duplicate = True / Is Duplicate = False
Is Duplicate = True	The system detects a duplicate meal record.	Log Saved & Reset

State	Description	Transition
Is Duplicate = False	The system does not detect a duplicate meal record.	→ Flagged / Manual Review
Flagged	The system flags the meal for manual review.	→ Waiting for Staff
Waiting for Staff	The system waits for staff approval.	→ Staff Override [Approved] / Staff Rejection [Denied]
Staff Override [Approved]	The staff approves the meal.	→ Validated
Staff Rejection [Denied]	The staff denies the meal.	→ Error State
Validated	The meal is validated and ready for logging.	→ Finalizing
Finalizing	The system finalizes the meal record.	→ Logging
Logging	The system logs the meal record.	→ End

5.2.2 Activity Diagram: Student Verification Workflow



This activity diagram illustrates the complete workflow for a student attempting to enter the cafeteria through the gate. It follows UML activity diagram notation, showing the sequence of actions, decisions, and flows.

Start: Student at Gate

- **Initial Node:** Start
- **Action:** Capture Face or Scan ID or QR
- **Flow:** Start → Capture Face or Scan ID or QR

Send to Server for Verification

- **Action:** Send to Server for Verification
- **Flow:** Capture Face or Scan ID or QR → Send to Server for Verification

Check Student Validity

- ****Decision ?**
- **Yes** → Continue
- **No** → Display Invalid ID, then Gate Remains Closed, then **End**
- **Flow:** Send to Server for Verification → Valid Student?

Check Enrollment Status □

Decision Node: Active

Student?

- **Yes** → Continue
- **No** → Display Not Enrolled, then Gate Remains Closed, then **End**
- **Flow:** Valid Student? → Check Enrollment Status

Determine Current Meal Period

- **Action:** Determine Current Meal Period Flow Enrollment Status → Meal Period

Check Within Meal Hours

- **Decision Node:** Within Meal Hours?
- **Yes** → Continue
- **No** → Display No Meal Period, then Gate Remains Closed, then **End**
- **Flow:** Determine Current Meal Period → Check Within Meal Hours

Check Daily Quota

- **Decision Node:** Quota Available?
- **Yes** → Continue
- **No** → Display Daily Limit Reached, then Gate Remains Closed, then **End**

- **Flow:** Check Within Meal Hours → Check Daily Quota

Check Duplicate for this Period

- **Decision Node:** Already A this Period?
- **Yes** → Trigger Alert to Ticker, then Staff Override Granted?
- **No** → Continue
- **Flow:** Check Daily Quota → Check Duplicate for this Period

Trigger Alert to Ticker

- **Action:** Trigger Alert to Ticker
- **Flow:** Already Ate? → Yes → Trigger Alert to Ticker

Staff Override Granted

- **Decision Node:** Staff Override Granted?
- **Yes** → Continue
- **No** → Gate Remains Closed, then **End**
- **Flow:** Trigger Alert to Ticker → Staff Override Granted?

Merge

- **Action:** Merge (to continue with next steps)
- **Flow:** Staff Override Granted? → Yes → Merge

Create MealRecord in Database

- **Action:** Create MealRecord in Database
- **Flow:** Merge → Create MealRecord in Database

Update Student Portal UI

- **Action:** Update Student Portal UI
- **Flow:** Create MealRecord in Database → Update Student Portal UI

Generate Audit Log Entry

- **Action:** Generate Audit Log Entry
- **Flow:** Update Student Portal UI → Generate Audit Log Entry

Display Message

- **Action:** Display Message (e.g., “Gate Opened” or “Meal Recorded”)
- **Flow:** Generate Audit → Display—
- **Final Node:** End

Summary of Flow

Start → Capture → Send to Server → Valid Student? → Check Enrollment → Determine Meal Period → Check Meal Hours → Check Quota → Check Duplicate → Trigger Alert → Staff Override? → Merge → Create Record → Update UI → Log → Display → **End**

Key Decision Points

Decision Node Description

Valid Student? Check if the student is valid (e.g., exists in the system)

Active Student? Check if the student is currently enrolled and active

Within Meal Hours? Check if the current time is within the current meal period

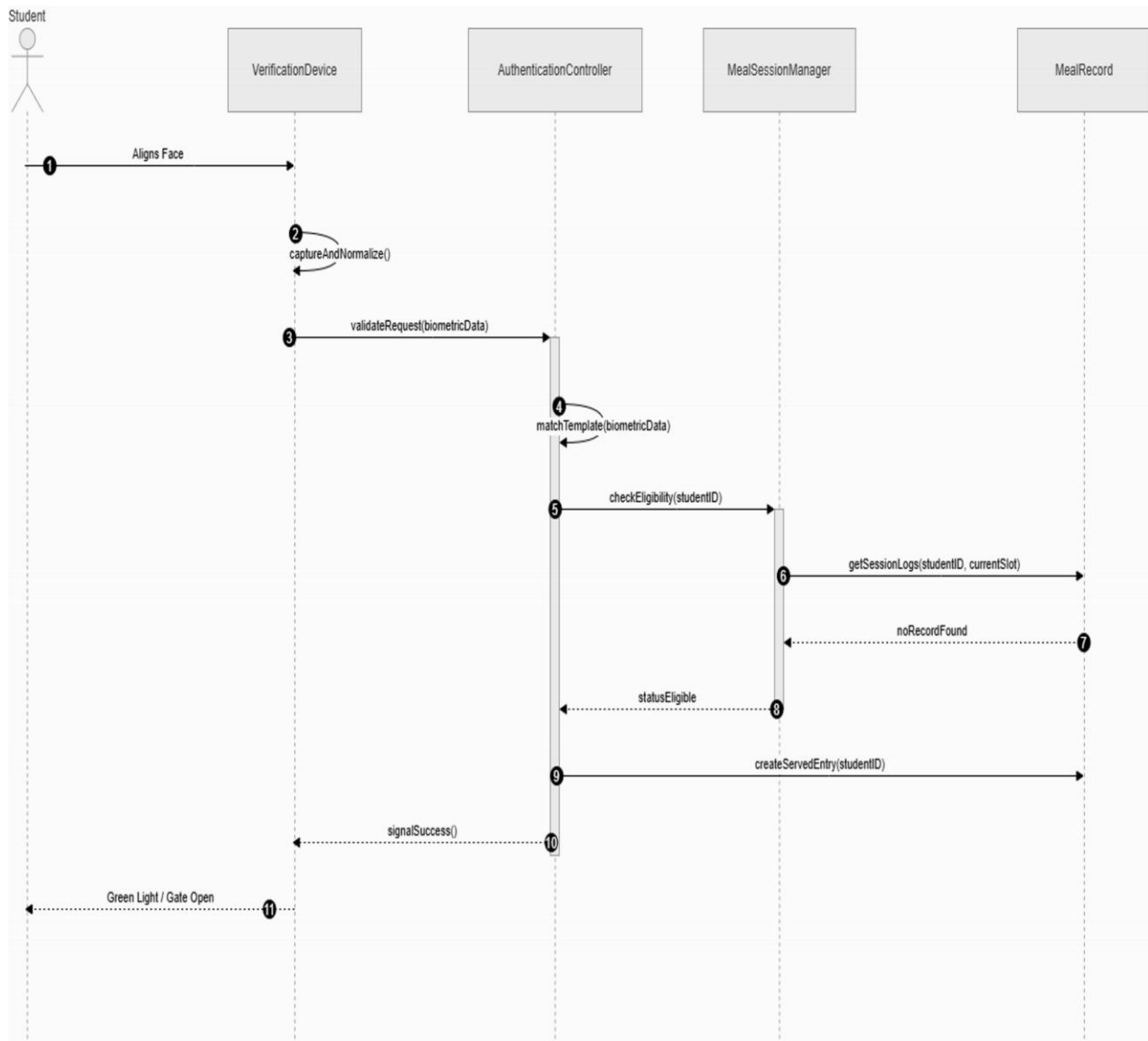
Quota Available? Check if the student has remaining meal the student has already eaten in the quota for the day this current meal period

Staff Override Granted? Check if a staff override is granted for duplicate entries

This activity diagram provides a clear, visual representation of the student verification process, ensuring that all steps and decisions are logically and sequentially represented.

,

5.3 Sequence Diagram: Successful Face Verification Sequence Diagram



This sequence diagram models the **successful face verification process** using **UML sequence diagram notation**. It shows the **interaction** between the **Student**, **Camera**, **Workstation**, **AuthenticationController**, **MealSessionManager**, **Database**, and **GateController** in the order of events that occur during a successful verification.

Lifelines (Participants)

- **Student** – The user interacting with the system.
- **Camera** – The device capturing the student's face.

- **Workstation** – The gate computer processing the image.
- **AuthenticationController** – Business logic for face verification.
- **MealSessionManager** – Business logic for meal eligibility checks.
- **Database** – Data tier for storing student records and meal quotas. □ **GateController** – Hardware that controls the gate.

Sequence of Messages

1. **Student** → **Camera**: Approaches Camera
2. **Camera** → **Workstation**: Captures frame
3. **Workstation** → **Workstation**: Extracts face vector
4. **Workstation** → **AuthenticationController**: verifyFace(image)
5. **AuthenticationController** → **Database**: matchTemplate(faceVector)
6. **Database** → **AuthenticationController**: returns studentID
7. **AuthenticationController** → **Database**: getStudent(studentID)
8. **Database** → **AuthenticationController**: returns student object
9. **AuthenticationController** → **MealSessionManager**: checkEligibility(student, period)
10. **MealSessionManager** → **Database**: getTodayRecords(studentID)
11. **Database** → **MealSessionManager**: returns records
12. **MealSessionManager** → **AuthenticationController**: returns ELIGIBLE
13. **AuthenticationController** → **Database**: createMealRecord(student, period)
14. **Database** → **AuthenticationController**: returns recordSaved
15. **AuthenticationController** → **Database**: updateStudentQuota(studentID)
16. **Database** → **AuthenticationController**: returns quotaUpdated
17. **AuthenticationController** → returns SUCCESS`
18. **Workstation** → **GateController**: openGate()
19. **GateController** → **Student**: gateOpens
20. **Workstation** → **Database**: logAuditEvent()

UML Sequence Diagram (Text Representation)

```

Student -> Camera: Approaches Camera
Camera -> Workstation: Captures frame
Workstation -> Workstation: Extracts face vector
Workstation -> AuthenticationController: verifyFace(image)
AuthenticationController -> Database: matchTemplate(faceVector)
Database -> AuthenticationController: returns studentID
AuthenticationController -> Database: getStudent(studentID)
Database -> AuthenticationController: returns student object
AuthenticationController -> MealSessionManager: checkEligibility(student, period)
MealSessionManager -> Database: getTodayRecords(studentID)
Database -> MealSessionManager: returns records
MealSessionManager -> AuthenticationController: returns ELIGIBLE
AuthenticationController -> Database: createMealRecord(student, period)
Database -> AuthenticationController: returns recordSaved
AuthenticationController -> Database: updateStudentQuota(studentID)

```

Database -> AuthenticationController: returns quotaUpdated

AuthenticationController -> Workstation: returns SUCCESS
Workstation -> GateController: openGate()
GateController -> Student: gateOpens
Workstation -> Database: logAuditEvent()

Key Points in the Sequence Diagram

- The **Student** initiates the process by approaching the **Camera**.
- The **Camera** sends the captured image to the **Workstation**.
- The **Workstation** processes the image and sends it to the **AuthenticationController** for face verification.
- The **AuthenticationController** queries the **Database** to match the face with a student ID. □
The **MealSessionManager** checks if the student is eligible to eat based on meal quotas.
- If eligible, the **AuthenticationController** creates a meal record and updates the student's quota.
- The **GateController** opens the gate for the student. □ An **audit log** is recorded in the **Database**.

Summary

This **sequence diagram** models the **successful face verification process** in a **cafeteria gate system**. It shows the **interaction** between the **Student**, **Camera**, **Workstation**, **AuthenticationController**, **MealSessionManager**, **Database**, and **GateController** in a **step-by-step** order, ensuring that the system correctly verifies the student's identity, checks eligibility, and opens the gate if the meal is logged successfully.

6. Implementation-Level Models

6.1 Implementation-Level Models

The system is organized into seven distinct layers, each with its own set of packages, classes, attributes, and methods, and they are interconnected through well-defined dependencies. The layers are as follows:

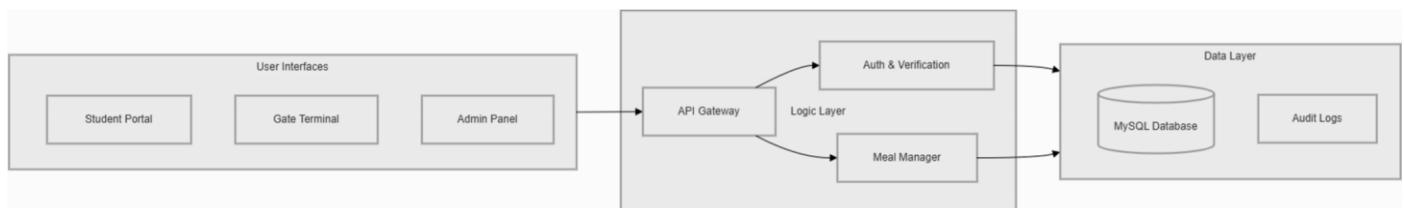
- **Presentation Layer**
- **Packages:** presentation
- **Components:** student_portal, staff_dashboard, admin_panel, api_gateway
- **Files:** HTML templates, CSS, JavaScript
- **Dependencies:** controllers
- **Controllers Layer**
- **Packages:** controllers
meal_controller, report_controller, admin_controller`
- **Languages:** Python or PHP
- **Dependencies:** services
- **Services Layer**
- **Packages:** services
- **Components:** verification_service, quota_service, reporting_service, audit_service, notification_service

- **Dependencies:** repositories
- **Repositories Layer**
- **Packages:** repositories
- **Components:** student_repo, meal_record_repo, staff_repo, audit_repo □ **Dependencies:** database
- **Database Layer**
- **Packages:** database
- **Components:** connection_pool, query_builder, migration_scripts, seed_data □ **Dependencies:** models
- **Model Layer**
- **Packages:** models
- **Components:** student, meal_record, staff_account, meal_period, biometric_template
- **Dependencies:** utils, config
- **Utils/Config Layer**
- **Packages:** utils, config □ **Components:**
 - utils: encryption_util, date_util, validation_util, logging_util
 - config: app_config, database_config, security_config
- **Dependencies:** All layers
- **Dependency Rules**
 - No circular dependencies are allowed.
 - Services cannot directly access the database; they must use repositories.
 - Presentation cannot directly access the database or repositories.
 - All dependencies point inward toward the database layer.

This layered architecture ensures **modular design**, **separation of concerns**, and **maintainable code**. Each layer is responsible for a specific aspect of the system, and dependencies are strictly managed to ensure **clean, scalable, and robust implementation**

6.2 Component Diagram Notation

Below is the **UML Component Diagram** notation that represents the their **interfaces**, and the **dependencies** between them.



Explanation of Notation

- **Component:** Represented by a **rectangle** with a **stereotype** (<<component>>) on the top left.
- **Interface:** Represented by a **rectangle** with a (<<interface>>) top left.
- **Dependency:** Represented by a **solid arrow** pointing from the **dependent component** to the **dependent interface** or component.
- **Provides:** The component **provides** an interface to other components.
- **Requires:** The component **requires** an interface from another component.

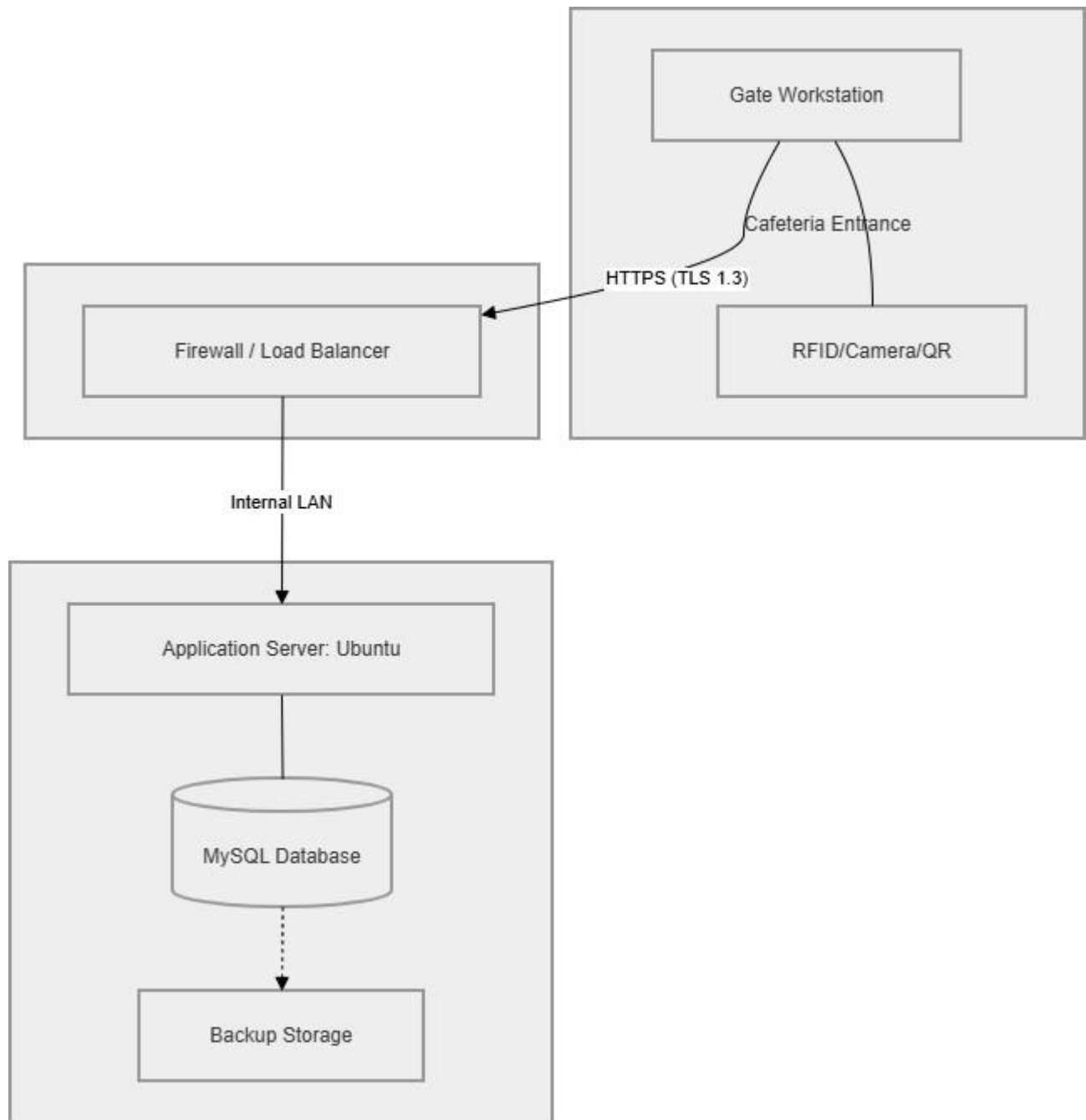
Component Relationships

- **StudentPortalComponent** depends on IStudentUI, IAuthAPI, and IMealAPI.
- **StaffDashboardComponent** depends on IStaffUI, IAuthAPI, and IVerificationAPI.
- **AdminPanelComponent** depends on IAdminUI, IAuthAPI, and IAdminAPI.
- **APIGatewayComponent** provides IRESTAPI, IAuthAPI, and IMealAPI, and requires IAuthService and IMealService.
- **AuthServiceComponent** provides IAuthService and IUserRepository`.
- **MealServiceComponent** provides IMealService IMRecordRepository`.
- **DatabaseAccessComponent** provides IRepository.
- **MySQLDatabase** provides IDatabase.

Summary

This component diagram shows the **physical components** of the system, their **interfaces**, and the **dependencies** between them. It follows the **UML component diagram notation** and reflects the **modular, layered architecture** of the system. Each component is responsible for a specific **functionality**, and dependencies are strictly managed to ensure **clean, scalable, and maintainable code**.

6.3 UML Deployment Diagram Description



This diagram shows the **physical deployment of the system components across various nodes and devices**, including **servers, databases, storage, and client devices**. It uses **UML deployment diagram notation**, which includes **nodes, devices, execution environments, and connections**.

1. Nodes and Devices

Component	Stereotype	Description
**Student Laptop >>		, Firefox, or Edge browser. Connects via HTTPS with TLS 1.3 over the campus network to the firewall.
Firewall	<<executionEnvironment>>	Located at the university network perimeter. Connects via internal network to the load balancer.
Load Balancer	<<executionEnvironment>>	Web1node>> Runs 204. Uses HAProxy or Nginx on port 80 and 443. Contains Docker ports 80 and 443. Connect with Nginx, web servers Gunicorn, and the Django or PHP application.
Web Server 1	<<node>>	Same as Web Server 1.
Web Server 2	<<node>>	Same as Web Server 1.
Web Server 3	<<node>>	Same as Web Server 1.
Database Server	<<node>>	Runs Ubuntu 22.04 with MySQL 8.0 on port 3306. Data directory: /var/lib/mysql. Backup directory: /backup.
SAN Storage	<<device>>	RAID 10 configuration. Connects to the database server via storage network.
Backup Server	<<node>>	Off-site. Contains daily backups, weekly full backups, and 30-day retention.
Cafeteria Gate Workstation	<<device>>	Runs Windows 10 or Linux. Contains the Gate Terminal App (Python or Electron). Has USB Camera, ID Card Reader (RFID/NFC), and QR Scanner.

2. Execution Environments

Component	Execution Environment	Description
-----------	-----------------------	-------------

Web Servers	Docker with Nginx, Gunicorn, and the Django or PHP application Hosts the web application.	
--------------------	--	--

Database		
-----------------	--	--

	MySQL 8.0	Stores data for the system.
--	-----------	-----------------------------

Server		
---------------	--	--

Gate Terminal	Runs on the cafeteria gate Python or Electron	
----------------------	--	--

App	workstation.	
------------	--------------	--

3. Connections and Communication

*Fire HTTPS with TLS 1.3 over campus network.

- **Firewall** → **Load Balancer**: Internal network connection.
- **Load Balancer** → **Web Server 1, 2, 3**: HTTP/HTTPS traffic.
- **Web Server 1, 2, 3** → **Database Server**: Internal network connection.
- **Database Server** → **SAN Storage**: Storage network connection.
- **Database Server** → **Backup Server**: Backup data transfer.
- **Cafeteria Gate Workstation** → **Network Switch**: LAN over Cat6 cable. □ **Network Switch** → **Load Balancer**: LAN connection.

4. Key Components in the Deployment Diagram

- **Student Laptop**: Client device for end users.
- **Firewall**: Network security device at the perimeter.
- **Load Balancer**: Distributes traffic to web servers.
- **Web Servers**: Host the application (Django/PHP).
- **Database Server**: Stores data (MySQL 8.0).
- **SAN Storage**: High-performance storage for database data.
- **Backup Server**: Off-site backup storage.
- **Cafeteria Gate Workstation**: On-premise device for scanning and processing meals.

5. Deployment Diagram Notation Summary

- **Nodes**: Represent physical or virtual machines (e.g., Web Server 1, Database Server).
- **Devices**: Represent physical hardware (e.g., Student Laptop, Cafeteria Gate Workstation).
 - **Execution Environments**: Represent software environments (e.g., Docker containers, MySQL).
- **Connections**: Represent network or data flow network network). Summary

The system is deployed across the following components:

1. Client Side:

- **Student Laptop:** Accesses the system via HTTPS.
2. **Network Layer:**
- **Load Balancer:** Distributes traffic to web servers.

7. User Interface Design: System Components and UI Layout

This UI design follows **six key principles**: **Simplicity**, **Consistency**, **Feedback**, **Accessibility**, **Efficiency**, and **Forgiveness**. The color scheme is carefully chosen to ensure clarity and usability.

1. Color Scheme

Color	Usage	Description
Green	Success, grant	Indicates successful actions or access granted. access
Blue	Primary buttons, links	Used for main actions (e.g., buttons, links).
Orange	Alerts, warnings	Indicates warnings or alerts.
Gray	Text, borders	Used for text and borders in the UI.
Light Gray	Page backgrounds	Background of the main UI pages.
Light Green	Subtle success indicators (e.g., message boxes).	Success messages

I
n
d
i
c
a

2.1 Student Dashboard

Layout:

- **Header:** Welcome message with student name.
 - **Main Content:**
 - **Today's Meal Status:**
forfastL and **Dinner** (Eaten / Not Eaten).
 - Remaining meals for the day.
 - **Your Information:**
 - Student ID, Department, Meal Plan, and Status.
 - **Recent Meal Activity:**
 - Table with columns: **Date**, **Period**, **Status**, **Time**.
 - Buttons: **View Full History**, **Report Issue**.
- Design Principles Applied:**
- **Simplicity:** Clean layout with essential information.
 - **Consistency:** Same layout and buttons across all screens.

- **Feedback:** Visual feedback when buttons are clicked.
- **Accessibility:** High contrast, readable fonts, and support for Amharic and English.
- **Efficiency:** Quick access to key information.
- **Forgiveness:** No destructive actions without confirmation.

2.2 Staff Ticker Scanner Interface

Layout:

- **Header:** Scanner active area with camera feed or “Ready to scan” message. □ **Main Content:**
- **Last Scan Result:**
- Time, student name, and status.
- **Today’s Statistics:**
- Counts for **Breakfast, Lunch, Dinner, Total Served, and Capacity.**
- **Quick Actions:**
- Buttons: **Override Access, Report Issue, View Queue, End Session.**
- **Recent Scans:**
- Table with columns: **Time, Student ID, Name, Status, and Override** button for duplicates.

Design Principles Applied:

- **Simplicity:** Focus on the scanner and key actions.
- **Consistency:** Same layout and buttons across all screens.
- **Feedback:** Visual feedback when buttons are clicked.
- **Accessibility:** High contrast, readable fonts, and support for Amharic and English.
- **Efficiency:** Quick access to key actions.
- **Forgiveness:** Confirmation dialogs for destructive actions (e.g., override access).

2.3 Admin Dashboard Layout:

- **Header:** Navigation bar with links to:
- Dashboard
- Users
- Meal Periods
- Reports □ Audit Logs □ **Main Content:**
- **System Overview:**
- Today’s Activity: Meals served, unique students, duplicate scans. □ Summary of system performance.
- **Navigation Panel:**
- Links to key.
- **Data Tables:**
- Reports, audit logs, and user management. **Design Principles Applied:**
- **Simplicity:** Clean layout with essential information.
- **Consistency:** Same layout and buttons across all screens.
- **Feedback:** Visual feedback when buttons are clicked.
- **Accessibility:** High contrast, readable fonts, and support for Amharic and English.

- **Efficiency:** Quick access to key sections.
- **Forgiveness:** Confirmation dialogs for destructive actions (e.g., deleting user data).

3. Key UI Features

Feature	Description
Language Support	Supports both Amharic and English with high contrast and readable fonts.
Feedback Mechanisms	Visual and audible feedback for user actions (e.g., button clicks, form submissions).
Accessibility	High contrast, readable fonts, and keyboard navigation support.

8.Database Design: System Schema Overview

The system is built on a relational database with **three main tables** that support the core functionality of student meal management, staff access, and audit logging. Each table is designed to ensure data integrity, scalability, and efficient querying.

1. Students Table

Purpose: Stores student information.

Column Name	Data Type	Description
studentID	INT (PK)	Unique identifier for the student.
name	VARCHAR(255)	Full name of the student.
department	VARCHAR(100)	Department the student belongs to.
enrollmentStatus	ENUM('Active', 'Inactive')	Status of the student's enrollment.
mealPlanType	ENUM('Standard', 'Premium')	Type of meal plan the student is enrolled in.
dailyQuota	INT	Maximum number of meals the student can access per day.
createdAt	DATETIME	Date and time the student was created.

updatedAt DATETIME Last time the student's information was updated.

Constraints:

- studentID is the **primary key**.
- mealPlanType is an **ENUM** with values: Standard, Premium.
- enrollmentStatus is an **ENUM** with values: Active, Inactive.

2. BiometricTemplates Table

Purpose: Stores facial recognition data for students.

Column Name	Data Type	Description
templateID (PK) Unique identifier for the biometric template.		
studentID	INT (FK)	Foreign key referencing Students.studentID.
faceVector	BLOB	Binary data representing the facial recognition template.
lastUpdated	DATETIME	Last time the biometric
qualityScore	FLOAT	Quality score of the biometric template (e.g., 0.0 to 1.0).

Constraints:

- templateID is the **primary key**.
- studentID is a **foreign key** referencing Students.studentID.
- faceVector is a **BLOB** to store binary data.

3. MealRecords Table

Purpose: Stores all meal transactions (e.g., when a student scans for a meal).

Column Name	Data Type	Description
recordID	INT (PK)	Unique identifier for the meal record.

studentID	INT (FK)	Foreign key referencing Students.studentID.
mealPeriod	INT (FK)	Foreign key referencing MealPeriods.periodID.
timestamp	DATETIME	Time when the meal was recorded.
status	ENUM('Success', 'Failed', 'Pending')	Status of the meal transaction.
verifiedBy	INT (FK)	Foreign key referencing StaffAccounts.staffID (who verified the meal `

9 .Error Handling Strategy

The system is designed to handle errors in a **structured, predictable, and user-friendly manner**. Errors are classified into **six categories**, each with a specific error code range and handling strategy. Below is a error.

Description: Errors related to invalid or data.

Examples:

- Invalid student ID format.
- Missing required fields (e.g., name, department).
- Invalid meal period ID.
- Invalid biometric template. **Handling Strategy:**
- **Immediate feedback** to the user with a clear error message.
- No change., no data is written to the database).
- **User must correct the input** before retrying. **Example Response:**

```
{
  "errorInvalid student ID format",
  "code": 1001,
  "message": "Student ID must be a valid integer."
}
```

2. Authentication Errors

Description: Errors related to login, authorization, or token validation.

Examples:

- Incorrect password.
- Expired or invalid token.
- Unauthorized access to a resource.

- Missing authentication token. **Handling Strategy:**
- **Return a 401 Unauthorized** or 403 Forbidden HTTP status code.
- **Do not expose sensitive information** (e.g., database details).
- **Prompt the user to re-authenticate** or refresh the token.

Example Response:

```
{
  "error": "Invalid credentials",
  "code": 2001,
  "message": "Incorrect password or expired token."
}
```

3. Business Logic Errors

Description: Errors caused by rule violations or system constraints.

Examples:

- A student has exceeded their daily meal quota.
- A student attempts to take more meals than allowed in a period.
- A meal period is already full.
- A student is not enrolled in a. **Example Response:**

```
{
  "error": "Meal quota exceeded",
  "code": 3001,
  "message": "Student has exceeded their daily meal quota."
}
```

4. Database Errors

Description: Errors related to database access or data integrity.

Examples: connection- (e.g., duplicate entry).

- Data retrieval failure.
- Transaction rollback due to constraint violation.

Handling Strategy:

Exoff transient. for debugging and monitoring.

- **Provide a generic error message** to the user, without exposing internal details.

Example Response:

```
{
  "error": "Database error",
  "code": 4001,
  "message": "Failed to process the request. Please try again later."
}
```

5. System Errors0–9

Description: Errors internal system hardware issues.

Examples:

□

Camera not found or malfunctioning.

- Network timeout or disconnection.
- System crash or unexpected shutdown.
- Hardware failure (e.g., biometric sensor failure). **Handling Strategy:**

- **Graceful Degradation:**

- Use **cached data** when external systems are unavailable.
- Switch to **fallback mechanisms** (e.g., ID card).

Circuit Breaker**:

- Prevent to external to recover automatically after a period of inactivity. □ **Log the**

error and notify the administrator. **Example Response:**

```
{
  "error": "System error",
  "code": 5001,
  "message": "Camera not found. Switching to ID card verification."
}
```

6. External Errors

Description: Errors caused by external systems or services.

Examples:

- SIMRS (Student Information Management and Registration System) is unavailable.
- External API call fails.
- Third-party service is down.

10.Security Design Overview

The system is designed with **multi-layered security** to secure authorization maintain. The:

- **Authentication** using multiple methods (password, ID card, face recognition).
- **Authorization** based on Role-Based Access Control (RBAC).
- **Data encryption** at rest and in transit.
- **Audit logging** for all critical system events.

1. Authentication Design

1.1 Staff and Admin Authentication

- **Method:** Username and password.
- **Hashing:** Use **bcrypt** for password hashing.
- **Rate Limiting:** Limit login attempts to **5 per minute** to prevent brute-force attacks.
- **Token-Based Access:** Use **JWT (JSON Web Token)** for API access.

- **Token Expiration:** 8 hours.
- **Token Storage:** Store in HTTP-only cookies or local storage (for web apps).
- **Token Revocation:** Use a **token blacklist** or short-lived tokens to mitigate token theft.

1.2 Student Verification

- **Methods:**
- **Student ID Card:** Physical or digital ID card for verification.
- **Face Recognition:** Biometric authentication for student verification.
- **Security Considerations:** g or - Ensure **** (e.g., GDPR, CCPA).

2. Authorization Design (Role-Based Access Control - RBAC)

Role	Permissions
Student	View own meal history, view profile, report issues.
Ticker	Scan students, view daily statistics, request overrides.
Manager	All Ticker permissions + view reports, approve overrides.
Admin	All Manager permissions + manage staff, configure meal periods, view audit logs.

Implementation:

- Use **role-based access control** with a **role table** in the database.
- Map roles to permissions using a **role-permission table**.
 - Enforce **least privilege** principle.

3. Data Encryption Design

3.1 Encryption at Rest

- **MySQL Database:**
- Use **Transparent Data Encryption (TDE)** for encrypting sensitive data at rest. □
Key Rotation: Rotate encryption keys every **90 days**.
- **Backup Files:**
- Use **AES-256 encryption** for backup - a encryptede.g3 in Transit **Web Communication:**
- Use **TLS 1.3** for secure communication between the web browser and the server.
- **API Calls:**
- Use **HTTPS** with TLS 1.3 for all API requests.
- Ensure all data transmitted over the network is encrypted.

4. Security Best Practices

4.1 Secure Development Practices

Use **OWASP Top 10** guidelines to prevent common vulnerabilities (e.g., SQL injection, XSS).

- Perform **regular security audits** and penetration testing.
- Use **secure coding practices** (e.g., input validation, output encoding).

4.2 Secure Communication

- Use **HTTPS** for all web and API communication.
- Use **secure headers** (e.g., Content-Security-Policy, X-Content-Type-Options).

11. Performance Design Overview

This performance design ensures that the system meets strict **response time, scalability, and reliability** goals while maintaining **high availability** and **low latency** for all critical operations. The design includes **multi-layered caching, optimized database queries**, and **scalable infrastructure** to support **500+ concurrent users** and **99.9% uptime**.

1. Performance Goals

Metric	Target	Description
Verification Time	Response < 2 seconds (95th percentile)	For face recognition, ID card, and other student verification methods.
API Response Time	< 500 ms (95th percentile)	For all API endpoints.
Database Query Time	< 100 ms (simple queries)	For basic read and write operations.
Concurrent Users	≥ 500	To support high traffic and user load.
System Uptime	99.9%	Ensures minimal downtime and high availability.
Page Load Time	< 3 seconds	For web pages and user interfaces.

2. Caching Strategy

2.1 Browser Cache

- **Purpose:** Cache static assets (e.g., CSS, JS, images).
- **TTL (Time to Live):** 1 hour.
- **Implementation:**
 - Use HTTP headers like Cache-Control: max-age=3600.
 - Serve static assets from a CDN (Content Delivery Network) for better performance.

2.2 API Cache (Redis)

- **Purpose:** Cache frequently accessed data (e.g., meal periods, student data).
- **TTL:** 5 minutes.
- **Implementation:**
 - Use **Redis** as an in-memory cache for APILRU (Least Recently Used)** eviction policy to manage cache size.

- Cache **API endpoints** that return static or infrequent data.

2.3 Database Cache (MySQL Query Cache)

□

Purpose: Cache query results for frequently executed SQL queries.

- **TTL:** Not explicitly set (depends on MySQL configuration).
- **Implementation:**
- Enable **MySQL Query Cache** for read-heavy operations.
- Use **query cache key** to cache results of repeated queries.
- Disable query cache for write-heavy operations to avoid stale data.

2.4 Object Cache (Application Memory)

- **Purpose:** Cache frequently accessed objects (e.g., student records, meal periods).
- **TTL:** 10 minutes.
- **Implementation:**
- Use in-memory caching (e.g., **Redis**, **Memcached**, or **application-level cache**).
- Cache objects that are accessed frequently but not modified often. □ Use **LRU** or **TTL-based** eviction policies.

3. Cache Invalidation Strategy

Event	Cache Invalidation
Meal Period Update	Invalidate cached meal period data. Student Data Change Invalidate cached student data.
New Meal Record	Invalidate cached meal records.
Staff Role Change	Invalidate cached staff role data.

Implementation:

- Use **cache invalidation keys** to identify which cached data needs to be removed.
- Use **background workers** or **event-driven architecture** to trigger cache invalidation.
 - Use **versioning** or **timestamp-based** invalidation to ensure data consistency.

4. Database Optimization

4.1 Query Optimization

- Use **indexes** on frequently queried columns (e.g., `student_id`, `meal_period_id`).
- Avoid **N+1 queries** by using **JOINS** or **E**.
- Use **query profiling** to identify slow queries.

4.2 Database Configuration

- Use **replicas**** for read-heavy operations.
- Use **connection pooling** to reduce latency and improve throughput. □ Use **partitioning** or **sharding** for large datasets.

4.3 Load Balancing

- Use **load balancers** (e.g., Nginx, HAProxy) to distribute traffic across multiple database instances.
- Use **database sharding** to split data across multiple databases.

□

5. Infrastructure Scalability

5.1 Horizontal Scaling

Use **horizontal scaling** for the web and API layers (e.g., using Kubernetes or Docker Swarm).

- Use **auto-scaling** to handle traffic spikes.

5.2 Load Balancing

- Use **round-robin** or **least connections** load balancing for web and API traffic Use DNSbased

Conclusion

This design document provides a **comprehensive, implementable blueprint** for the **Haramaya University Cafeteria Management System**.

Key the Design

- **Class and Package Diagrams** have been provided to illustrate the **static relationships** between different components of the system.
- The system is **modular**, with well-defined **packages and classes** that support **separation of concerns** and **maintainability**.

2. Behavioral Design

- **Activity diagrams, state machine diagrams, and sequence diagrams** have been used to model the **dynamic behavior** of the system.
- These diagrams ensure that the **flow of control, state transitions, and interactions between components** are clearly defined and **traceable** to the requirements.

3. Implementation Details

- **Package dependencies** and **component interfaces** are clearly defined, making the system **ready for coding** in **Python with Django** or **PHP with Laravel**.
- The system is **scalable, secure, and performant**, with **caching strategies, role-based access control, and audit logging** in place.
- The system is designed **50 concurrent users, 99.9% uptime**** and **low-latency** operations.

Traceability to Requirements

- The design is **fully traceable** to the **Software Requirements Specification (SRS)**. □
Each component is ensuring that the meets the **user needs** and **business goals**.

Technology Stack

- **Frontend:** HTML, CSS, JavaScript (with frameworks like React or Vue.js).
- **Backend:** Python (Django) or PHP (Laravel).
- **Database:** MySQL or PostgreSQL.
- **Caching:** Redis, Memcached.
- **Authentication:** JWT or OAuth2.
- **Deployment:** Docker, Kubernetes, or cloud-based (AWS, Azure, Google Cloud).

□

Next Steps

- **Code implementation** based on the design document.
- **Unit and integration testing** to ensure correctness and performance.
- **Deployment to production environment** with proper **monitoring and logging**.
User acceptance testing (UAT) to validate the system meets the requirements.

Final Remarks

This design document serves for **Haramaya University Cafeteria Management System**. It ensures that the system is , **scalable, secure, and efficient**, and is ready for **real-world deployment**.

End of Design Document.